

B 2.4.3 Bubble and Selection sort



2.4.3 Construct and trace algorithms to implement bubble sort and selection sort, evaluating their time and space complexities.

Master the basics of sorting. This section explores two fundamental sorting algorithms: bubble sort and selection sort. Learn how to construct, trace, and implement these algorithms in Java and Python. Analyse their efficiency with Big O notation and understand how their performance compares in different scenarios.

Last updated 04/10/2025

Learning Objectives

- Students can explain the mechanisms of bubble sort and selection sort algorithms.
- Students can construct and trace the execution of bubble sort and selection sort algorithms for given inputs.
- Students can implement bubble sort and selection sort algorithms in Java and Python.
- Students can analyse the complexity of these algorithms' time and space using Big O notation.
- Students can compare the efficiency of bubble and selection sort in different scenarios.

IB ATL Skills:

Thinking: Critical thinking, creative thinking, transfer

Research: Information literacy

Communication: Communication, collaboration

Self-Management: Organization, time management, reflection

Lesson Plan (2 hours)

Part 1: Bubble Sort (1 hour)

Engage (10 minutes)

Real-world scenario: Show a visual representation of unsorted items (e.g., playing cards, numbered balls). Ask students how they would sort these items in ascending or descending order.

Discussion: Facilitate a brief discussion about different sorting strategies and lead into the concept of bubble sort.

Explore (20 minutes)

Introduce bubble sort: Explain the algorithm as a simple sorting method that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order.

Visual Aid: Use a diagram or animation to illustrate the step-by-step process of bubble sorting, emphasizing the "bubbling up" of the largest (or smallest) element to its correct position.

Explain (15 minutes)

Implementation: Demonstrate how to implement bubble sort in both Java and Python. Provide code examples and explain the logic clearly, including using nested loops and the swapping mechanism.

Tracing: Work through a few examples with different inputs, tracing the algorithm's execution step by step.

Elaborate (10 minutes)

Efficiency Analysis: Discuss the time complexity of bubble sort ($O(n^2)$). Explain that the algorithm needs to perform nested iterations in the worst and average cases.

Space Complexity: Explain that bubble sort is an in-place sorting algorithm with $O(1)$ space complexity.

Evaluate (5 minutes)

Quick Quiz: Ask short-answer questions or provide code snippets for students to analyze and determine bubble sort's time and space complexity.

Part 2: Selection Sort (1 hour)

Engage (10 minutes)

Real-world scenario: Show a visual representation of unsorted items. Ask students to think about a sorting strategy where they repeatedly find the smallest (or largest) element and place it in its correct position.

Discussion: Discuss the idea of selection and lead into the concept of selection sort.

Explore (20 minutes)

Introduce selection sort: Explain the algorithm as a sorting method that divides the input list into two parts: the sorted sublist, which is built up from left to right, and the unsorted sublist, which is the remaining part.

Visual Aid: Use a diagram or animation to illustrate the step-by-step process of selection sort, highlighting the finding of the minimum element and swapping it with the first element of the unsorted sublist.

Explain (15 minutes)

Implementation: Demonstrate how to implement selection sort in both Java and Python. Provide code examples and explain the logic clearly, including using nested loops and finding the minimum element.

Tracing: Work through a few examples with different inputs, tracing the algorithm's execution step by step.

Elaborate (10 minutes)

Efficiency Analysis: Discuss the time complexity of selection sort ($O(n^2)$). Explain that it always has nested iterations regardless of the initial order of the list.

Space Complexity: Explain that selection sort is also an in-place sorting algorithm with $O(1)$ space complexity.

Evaluate (5 minutes)

Worksheet: Provide a worksheet with code snippets or algorithm descriptions for students to analyze and determine the selection sort's time and space complexity.

Lesson Materials

Presentations

[Link to presentation](#)

[Link to presentation](#)

eBook

The following eBook pages can be used by students to revise and review the topic.

[B 2.4.3 Bubble Sort and Selection Sort in Java](#)

[B 2.4.3 Bubble Sort and Selection Sort in Python](#)

Worksheets

The following worksheets can be used during each lesson to test and develop students' understanding of the topic.

B 2.4.3 Worksheet Java

B 2.4.3 Worksheet Python

B 2.4.3 Worksheet Java Answers

B 2.4.3 Worksheet Python Answers

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**